

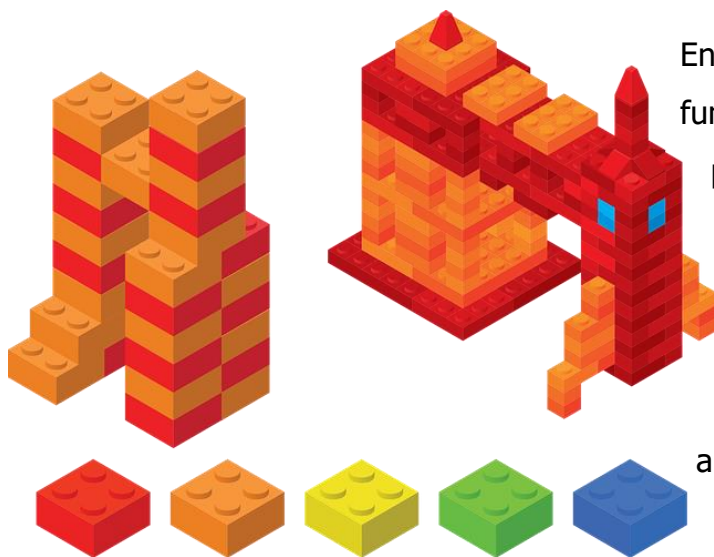
Unidad Didáctica

**Programación Estructurada,
Unidad IV**

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	4
Introducción a los subalgoritmos o subprogramas	4
Criterios de descomposición modular.....	4
Clasificación de subprogramas	5
Procedimiento (subrutinas).....	5
Funciones.....	7
Ámbito: Variables locales y globales	8
Comunicación con subprogramas: paso de parámetros.....	10
Paso de parámetros	10
Paso por valor	10
Paso por referencia.....	11
Comparaciones de los métodos de paso de parámetros.....	11
Funciones y procedimientos como parámetros	13
Los efectos laterales	14
Ventajas de los subprogramas	14
Bibliografía y Webgrafía	15
Glosario	15

Introducción



En esta unidad, exploraremos los fundamentos esenciales de la programación estructurada, centrándonos en la creación y utilización efectiva de subprogramas o subalgoritmos.

Nuestro objetivo principal es dotar a los estudiantes de las habilidades necesarias para descomponer

problemas complejos en unidades más pequeñas y manejables, facilitando así su resolución de manera modular y eficiente. Los objetivos específicos de esta unidad incluyen la comprensión y aplicación de las normas sintácticas básicas para la declaración de módulos, la diferenciación entre el uso de variables globales y locales en los subprogramas, y el manejo adecuado de parámetros de variables y de valor. Además, exploraremos cómo convertir las subtarefas de un problema en procedimientos o funciones, y cómo integrar estos conocimientos en la resolución de situaciones reales.

Exploraremos temas clave como la introducción a los subalgoritmos o subprogramas, los criterios de descomposición modular, la clasificación de subprogramas en procedimientos y funciones, y el ámbito de las variables locales y globales. También analizaremos la comunicación con subprogramas a través del paso de parámetros, los distintos métodos de paso de parámetros, y las ventajas de utilizar subprogramas en el desarrollo de software. A lo largo de esta unidad, animaremos a los estudiantes a articular los valores morales, cívicos y éticos en su desempeño académico, reconociendo la importancia de la responsabilidad y la integridad en el ámbito de la programación.

Introducción a los subalgoritmos o subprogramas

Los subalgoritmos o subprogramas son piezas fundamentales en la programación estructurada, ya que nos permiten dividir un problema complejo en partes más pequeñas y manejables. Imagina que estás cocinando una receta complicada: en lugar de intentar hacer todo al mismo tiempo, dividirías las tareas en pasos más simples, como picar vegetales, cocinar la carne, y preparar la salsa.

En PSeInt, podemos crear subprogramas utilizando la palabra clave función seguida del nombre del subprograma y sus parámetros, y finalizamos con finfuncion.

```
// Subprograma para sumar dos números
Funcion suma = sumar(num1,num2) //delaracion y definicion de la funcion
    // Funcion variable_de_retorno = nombreDeLaFuncion (lista de parametros
separados por coma)
        Definir suma Como Entero;
        suma = num1 + num2;
FinFuncion

Algoritmo suma_numeros
    Definir num1, num2, resultado Como Entero;
    Escribir "Ingresa el primer número:";
    Leer num1;
    Escribir "Ingresa el segundo número:";
    Leer num2;
    resultado = sumar(num1, num2);// Llamada al subprograma y obtención del
resultado
    Escribir "La suma es:", resultado;
FinAlgoritmo
```

Criterios de descomposición modular

La descomposición modular es un proceso clave en la programación estructurada que consiste en dividir un problema en módulos o subprogramas más pequeños y manejables. Al descomponer un problema en partes más simples, podemos abordar cada una por separado, lo que facilita la comprensión y resolución del problema en su conjunto.

Existen varios criterios que podemos seguir para descomponer un problema de manera modular:

- **Funcionalidad coherente:** Cada módulo debe tener una función específica y coherente. Por ejemplo, si estamos desarrollando un programa de gestión de inventario, podríamos tener módulos para agregar productos, eliminar productos y generar informes.
- **Reutilización de código:** Identificar partes del código que pueden ser reutilizadas en diferentes partes del programa y convertirlas en subprogramas. Esto nos ayuda a evitar la repetición de código y a mantener nuestro programa más organizado.
- **Abstracción:** Al descomponer un problema en módulos, debemos enfocarnos en la funcionalidad del módulo sin preocuparnos por los detalles de implementación. Esto nos permite cambiar la implementación interna de un módulo sin afectar otras partes del programa.

En PSeInt, podemos aplicar estos criterios al diseñar nuestros programas, creando subprogramas que realizan tareas específicas y luego combinándolos para resolver el problema completo.

Clasificación de subprogramas

Los subprogramas son segmentos de código que realizan una tarea específica dentro de un programa más grande. En PSeInt, podemos clasificar los subprogramas en dos categorías principales: procedimientos y funciones.

Procedimiento (subrutinas)

Los procedimientos, también conocidos como subrutinas, son subprogramas que realizan una tarea sin devolver un valor. Es decir, ejecutan un conjunto de instrucciones pero no producen un resultado que pueda ser utilizado en otras partes del programa. Son útiles cuando necesitamos realizar acciones como mostrar

mensajes en pantalla, actualizar datos, o realizar cálculos sin necesidad de devolver un valor.

```
Algoritmo mostrar_mensaje
    // Llamada al procedimiento para mostrar un mensaje
    mostrarMensaje("¡Hola mundo!");
FinAlgoritmo

// Procedimiento para mostrar un mensaje en pantalla
Funcion  mostrarMensaje(mensaje)
    Escribir mensaje;
FinFuncion
```

Sustitución de argumentos/parámetros

La sustitución de argumentos o parámetros se refiere al proceso de proporcionar valores concretos a los parámetros de un subprograma al llamarlo o invocarlo. Estos valores concretos, conocidos como argumentos, son pasados al subprograma y se utilizan en lugar de los parámetros definidos en su declaración.

En PSeInt, cuando llamamos a un subprograma y le proporcionamos valores para sus parámetros, estos valores se asignan a las variables definidas como parámetros dentro del subprograma. Esto permite que el subprograma trabaje con datos específicos sin necesidad de conocer los valores exactos de antemano.

Veamos un ejemplo simple para entender mejor cómo funciona la sustitución de argumentos en PSeInt:

```
Algoritmo main
    Definir numero1, numero2 Como Entero;
    numero1 = 5;
    numero2 = 3;
```

```

        // Llamamos al subprograma 'sumarNumeros' y pasamos 'numero1' y 'numero2' como
argumentos
        sumarNumeros(numero1, numero2);
FinAlgoritmo

// Subprograma para sumar dos números
Funcion sumarNumeros(num1, num2)
    Definir resultado Como Entero;
    resultado = num1 + num2;
    Escribir "El resultado de la suma es:", resultado;
FinFuncion

```

Funciones

Las funciones son subprogramas que realizan una tarea y devuelven un valor como resultado. A diferencia de los procedimientos, las funciones pueden ser utilizadas para realizar cálculos y retornar un resultado que puede ser utilizado en otras partes del programa.

Declaración de funciones

```

Algoritmo calcular_promedio
    Definir calificacion1, calificacion2, calificacion3, promedio Como Real;

    calificacion1 = 0;
    calificacion2 = 0;
    calificacion3 = 0;
    promedio = 0;

    Escribir "Ingrese la primera calificación:";
    Leer calificacion1;
    Escribir "Ingrese la segunda calificación:";
    Leer calificacion2;
    Escribir "Ingrese la tercera calificación:";
    Leer calificacion3;

    // Llamada a la función para calcular el promedio
    promedio = calcularPromedio(calificacion1, calificacion2, calificacion3);
    Escribir "El promedio es:", promedio;
FinAlgoritmo

```

```
// Función para calcular el promedio de tres calificaciones
Funcion promedio = calcularPromedio(calificacion1, calificacion2, calificacion3)
    Definir promedio Como Real;
    promedio = (calificacion1 + calificacion2 + calificacion3) /
3;
FinFuncion
```

En este ejemplo, creamos una función llamada `calcularPromedio` que recibe tres calificaciones como parámetros y devuelve el promedio de ellas. Luego, en el algoritmo principal, llamamos a esta función para calcular el promedio de las calificaciones ingresadas por el usuario.

Invocación o llamada a las funciones

La invocación o llamada a una función se realiza utilizando su nombre seguido de los parámetros entre paréntesis. El valor devuelto por la función puede ser asignado a una variable o utilizado directamente en el programa.

Es importante recordar que al llamar a una función, debemos proporcionar los argumentos necesarios en el mismo orden en el que fueron definidos en la declaración de la función

Ámbito: Variables locales y globales

El ámbito se refiere al alcance o la visibilidad que tienen las variables en un programa. En la programación estructurada, es importante entender la diferencia entre las variables locales y globales, ya que esto afecta cómo se pueden acceder y modificar las variables en diferentes partes del código.

Variables locales:

Las variables locales son aquellas que se definen dentro de un subprograma y solo son accesibles dentro de ese subprograma. Esto significa que su ámbito está limitado al bloque de código en el que fueron declaradas. Las variables locales se crean

cuando el subprograma comienza a ejecutarse y se destruyen cuando el subprograma termina su ejecución.

Veamos un ejemplo para ilustrar el ámbito de las variables locales:

```
Algoritmo ejemplo
    // Declaración de variables locales
    Definir numero Como Entero;
    numero = 10;
    Escribir "El número dentro del subprograma es:", numero;

    // Llamada al subprograma
    mostrarNumero(5);
FinAlgoritmo

// Subprograma para mostrar el valor de una variable local
Funcion mostrarNumero(k)
    Definir numero Como Entero;
    numero = k;
    Escribir "El número dentro del subprograma es:", numero;
FinFuncion
```

En este ejemplo, la variable `numero` se declara tanto en el algoritmo principal como en el subprograma `mostrarNumero`. Sin embargo, son variables diferentes con el mismo nombre, ya que están en ámbitos diferentes. La variable `numero` dentro del algoritmo principal es una variable local para ese algoritmo, mientras que la variable `numero` dentro del subprograma `mostrarNumero` también es una variable local para ese subprograma.

Variables globales:

Las variables globales son aquellas que se definen fuera de cualquier subprograma y son accesibles desde cualquier parte del código. Su ámbito se extiende a lo largo de todo el programa, lo que significa que pueden ser utilizadas y modificadas en cualquier parte del código, incluidos los subprogramas.

No se pueden declarar variables globales en PseInt

Comunicación con subprogramas: paso de parámetros

En la programación, a menudo necesitamos comunicarnos con subprogramas para pasarles datos y recibir resultados. Esto se logra mediante el paso de parámetros, que son valores o referencias a variables que se envían desde el algoritmo principal o desde otros subprogramas hacia el subprograma en cuestión.

Paso de parámetros

El paso de parámetros se refiere al proceso de enviar valores o referencias a variables a un subprograma para que pueda realizar su tarea. En PSeInt, podemos pasar parámetros de dos formas diferentes: por valor y por referencia.

Paso por valor

Cuando pasamos un parámetro por valor, lo que se envía al subprograma es una copia del valor de la variable. Esto significa que cualquier modificación realizada dentro del subprograma no afectará a la variable original en el algoritmo principal.

Veamos un ejemplo de paso por valor:

```
Algoritmo ejemplo
    Definir numero Como Entero;
    numero = 5;

    // Llamada al subprograma con paso por valor
    duplicarNumero(numero);
    Escribir "El número original es:", numero;
FinAlgoritmo

// Subprograma que duplica un número (paso por valor)
Funcion duplicarNumero(num)
    num = num * 2;
    Escribir "El número duplicado es:", num;
FinFuncion
```

En este ejemplo, el valor de numero se pasa por valor al subprograma duplicarNumero. Aunque dentro del subprograma se duplica el valor de num, esto no afecta al valor original de numero en el algoritmo principal.

Paso por referencia

Cuando pasamos un parámetro por referencia, lo que se envía al subprograma es la dirección de memoria de la variable. Esto significa que cualquier modificación realizada dentro del subprograma afectará directamente a la variable original en el algoritmo principal.

Veamos un ejemplo de paso por referencia:

```
Algoritmo ejemplo
  Definir numero Como Entero;
  numero = 5;

  // Llamada al subprograma con paso por referencia
  duplicarNumeroPorReferencia(numero);
  Escribir "El número duplicado es:", numero;
FinAlgoritmo

// Subprograma que duplica un número (paso por referencia)
Funcion duplicarNumeroPorReferencia(num Por Referencia)
  num = num * 2;
  Escribir "El número duplicado es:", num;
FinFuncion
```

En este ejemplo, la variable `numero` se pasa por referencia al subprograma `duplicarNumeroPorReferencia`. Como resultado, cualquier modificación realizada en `num` dentro del subprograma afectará directamente al valor de `numero` en el algoritmo principal.

Comparaciones de los métodos de paso de parámetros

La elección entre paso por valor y paso por referencia depende de las necesidades específicas del programa y del comportamiento deseado. El paso por valor se utiliza cuando queremos evitar modificar la variable original en el algoritmo principal, mientras que el paso por referencia se utiliza cuando queremos que los cambios realizados dentro del subprograma afecten directamente a la variable original. Es importante tener en cuenta estos aspectos al diseñar nuestros programas para

asegurarnos de utilizar el método de paso de parámetros más adecuado en cada situación.

	<i>Paso por valor</i>	<i>Paso por referencia</i>
<i>Concepto</i>	Se pasa una copia del valor de la variable.	Se pasa la dirección de memoria de la variable.
<i>Modificación de la variable original</i>	No se afecta, ya que se trabaja con una copia del valor.	Se ve afectada directamente, ya que se trabaja con la variable original.
<i>Uso de memoria</i>	Utiliza más memoria, ya que se crea una copia de la variable.	Utiliza menos memoria, ya que se pasa la dirección de memoria.
<i>Impacto en el rendimiento</i>	Puede tener un impacto en el rendimiento si se pasan grandes cantidades de datos, debido a la necesidad de copiarlos.	Generalmente más eficiente, especialmente para grandes cantidades de datos, ya que no se realizan copias.
<i>Protección de la variable original</i>	Ofrece protección, ya que no se puede modificar la variable original desde el subprograma.	Menos protección, ya que cualquier modificación en la variable dentro del subprograma afectará directamente a la variable original.

Funciones y procedimientos como parámetros

En la programación, es posible utilizar funciones y procedimientos como parámetros en otros subprogramas. Esto nos permite crear subprogramas más flexibles y genéricos, capaces de adaptarse a diferentes situaciones según las necesidades del programa.

Funciones como Parámetros

Las funciones pueden ser utilizadas como parámetros en otros subprogramas. Esto significa que podemos pasar una función como argumento a otro subprograma, lo que nos permite realizar operaciones específicas en función de la función pasada como parámetro.

Veamos un ejemplo de cómo utilizar una función como parámetro:

Algoritmo ejemplo

```
Definir num Como Entero;  
num = 5;  
// Llamada al subprograma con una función como parámetro  
operacionPersonalizada(num, duplicarNumeroPorReferencia(num));
```

FinAlgoritmo

```
// Subprograma que realiza una operación personalizada con una función como  
parámetro
```

```
Funcion operacionPersonalizada(num, duplicado)
```

```
    Escribir "el numero es ", num;
```

```
    Escribir "el numero duplicado es ", duplicado;
```

FinFuncion

```
// Función que duplica un número (paso por referencia)
```

```
Funcion num = duplicarNumeroPorReferencia(num)
```

```
    num = num * 2;
```

FinFuncion

Procedimientos como Parámetros

En muchos lenguajes los procedimientos y funciones son tratados indistintamente, sin embargo, en nuestro ejercicio los procedimientos no devuelven valor, por ende, no se pueden usar como parámetro.

Los efectos laterales

Cuando utilizamos funciones o procedimientos como parámetros, es importante tener en cuenta los efectos laterales que pueden producirse. Los efectos laterales ocurren cuando una función o procedimiento modifica estados fuera de su ámbito local, lo que puede afectar el comportamiento del programa en otros lugares.

Ventajas de los subprogramas

1. **Reutilización de código:** Los subprogramas permiten encapsular ciertas funcionalidades en bloques de código independientes y reutilizables. Esto significa que una vez que se ha escrito y probado un subprograma, puede ser llamado y utilizado en múltiples partes del programa sin necesidad de volver a escribir el mismo código.
2. **Modularidad:** La división del código en subprogramas facilita la comprensión y el mantenimiento del software. Cada subprograma puede centrarse en una tarea específica y, por lo tanto, el código se vuelve más organizado y fácil de entender. Además, los cambios realizados en un subprograma no afectarán a otros, lo que facilita la corrección de errores y la realización de actualizaciones.
3. **Abstracción:** Los subprogramas permiten abstraer detalles de implementación complejos detrás de una interfaz simple y fácil de entender. Esto significa que los usuarios del subprograma solo necesitan conocer cómo utilizarlo correctamente, sin necesidad de entender cómo funciona internamente.

4. **Facilita la depuración:** Al dividir el código en subprogramas más pequeños y manejables, se facilita la identificación y corrección de errores. Los subprogramas más pequeños son más fáciles de probar y depurar, lo que ayuda a reducir la complejidad del proceso de depuración.
5. **Facilita la colaboración:** El uso de subprogramas facilita la colaboración entre múltiples programadores en un proyecto de software. Cada programador puede trabajar en diferentes subprogramas de manera independiente, lo que permite una división del trabajo más eficiente y una integración más sencilla de las diferentes partes del programa.
6. **Mejora la legibilidad del código:** La división del código en subprogramas con nombres descriptivos ayuda a mejorar la legibilidad del código. Los nombres de los subprogramas pueden proporcionar pistas sobre lo que hace cada parte del código, lo que facilita su comprensión para otros programadores y para uno mismo en el futuro.

Bibliografía y Webgrafía

Steven S. Skiena, Miguel A. Revilla, et ál. Desafíos de programación: El manual de entrenamiento para concursos de programación (Spanish Edition) OJBooks (2020)
Vickler A. Algoritmos. Ladoo Publishing LLC (2022)
Pes C. Pseudocódigo para principiantes. Independently published (27 Abril 2022)

Glosario

- **Ámbito:** Alcance o la visibilidad que tienen las variables en un programa.
- **Comunicación con subprogramas:** Proceso de enviar datos a subprogramas y recibir resultados.
- **Paso de parámetros:** Proceso de enviar valores o referencias a variables a un subprograma para que pueda realizar su tarea.
- **Paso por referencia:** Método de paso de parámetros en el que se envía la dirección de memoria de la variable.
- **Paso por valor:** Método de paso de parámetros en el que se envía una copia del valor de la variable.
- **Procedimiento** (subrutinas): Subprogramas que realizan una tarea sin devolver un valor.
- **Subprograma:** Segmento de código que realiza una tarea específica dentro de un programa más grande.
- **Variables globales:** Variables definidas fuera de cualquier subprograma y accesibles desde cualquier parte del código.
- **Variables locales:** Variables definidas dentro de un subprograma y solo accesibles dentro de ese subprograma.